

Greedy Algorithm Patterns by Aman

A **Greedy Algorithm** builds a solution piece by piece, always choosing the next piece that offers the most immediate benefit (the "locally optimal choice"). The hope is that these local choices will lead to a **globally optimal** solution.

Pattern 1: Interval Scheduling (Activity Selection)

This is the most classic greedy pattern. You have a set of activities with start and end times, and you want to perform as many as possible without them overlapping.

- **Why we use it:** To maximize throughput or resource utilization when resources are mutually exclusive (e.g., one meeting room).
- **How to identify:**
 - Input involves intervals (start, end).
 - Goal: "Maximize the number of non-overlapping items."
- **The Strategy:** Always pick the activity that **finishes earliest**. This leaves the maximum amount of time for remaining activities.
- **Example:** given `[(1,3), (2,4), (3,5)]`, we pick `(1,3)`, skip `(2,4)` because it overlaps, and pick `(3,5)`.

Java

```
// 1. Sort by End Time
Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));

int count = 1;
int lastEndTime = intervals[0][1];

for (int i = 1; i < intervals.length; i++) {
    // 2. If current start time >= last end time, select it
    if (intervals[i][0] >= lastEndTime) {
        count++;
        lastEndTime = intervals[i][1];
    }
}
```

Pattern 2: The Optimal Merge Pattern (Two-Way Merge)

This pattern is used when combining items comes with a cost that accumulates, such as connecting ropes or merging files.

- **Why we use it:** To minimize the total cost of repeated operations where the cost depends on the size of the items involved.
- **How to identify:**
 - Problem asks for "minimum cost to connect/merge all elements."
 - Merging two items creates a new item with a combined size.
- **The Strategy:** Always merge the **two smallest** items currently available. This ensures the smallest values contribute to the total cost the most number of times (depth in the merge tree).
- **Example (Connect Ropes):** Ropes `[4, 3, 2, 6]`.
 1. Sort/Heap: `[2, 3, 4, 6]`
 2. Merge 2+3=5. Cost=5. New list: `[4, 5, 6]`
 3. Merge 4+5=9. Cost=5+9=14. New list: `[6, 9]`
 4. Merge 6+9=15. Total Cost=14+15=29.

Java

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// Add all elements to heap...

int totalCost = 0;
while (minHeap.size() > 1) {
    // 1. Extract two smallest
    int first = minHeap.poll();
    int second = minHeap.poll();

    // 2. Accumulate cost and add sum back
    int currentCost = first + second;
    totalCost += currentCost;
    minHeap.add(currentCost);
}
```

Pattern 3: Value-to-Weight Ratio (Fractional Knapsack)

Unlike the 0/1 Knapsack (which is Dynamic Programming), here you can take *parts* of an item.

- **Why we use it:** To get the maximum value for a limited capacity when items are divisible.
- **How to identify:**
 - Items have a **Value** and a **Cost/Weight**.
 - You have a limited capacity.
 - You are allowed to break items (e.g., gold dust, liquids).
- **The Strategy:** Calculate the **Value per Unit Weight** for each item. Sort items from highest ratio to lowest. Take as much as possible of the highest ratio item, then the next, until full.
- **Example:** Item A (Val 60, Wt 10, Ratio 6), Item B (Val 100, Wt 20, Ratio 5). Capacity 30. Take all of A (10wt), take all of B (20wt).

Java

```
class Item { double value, weight, ratio; }

// 1. Sort by Ratio (Descending)
Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));

double totalValue = 0;
for (Item i : items) {
    if (capacity >= i.weight) {
        capacity -= i.weight;
        totalValue += i.value;
    } else {
        // Take fraction of the remaining
        totalValue += (i.value / i.weight) * capacity;
        break; // Knapsack is full
    }
}
```

Pattern 4: Two Pointers / Sorting (Partitioning)

Sometimes greedy logic is applied simply by sorting the array and matching the smallest available resource to the smallest demand (or largest to largest).

- **Why we use it:** To satisfy requirements using the minimal efficient resources.
- **How to identify:**
 - Matching two arrays (e.g., People vs. Boats, Children vs. Cookies).
 - Constraints like "one boat holds at most 2 people."
- **The Strategy:** Sort both arrays. Use two pointers to find the "best fit" match.
- **Example (Boats to Save People):** Sort people by weight. Try to pair the heaviest person with the lightest person. If they fit, great (2 people saved). If not, the heaviest person goes alone (1 person saved).

Java

```
Arrays.sort(people);
int left = 0, right = people.length - 1;
int boats = 0;

while (left <= right) {
    // If lightest and heaviest fit together
    if (people[left] + people[right] <= limit) {
        left++; // Account for the lightest
    }
    right--; // Heaviest always gets on a boat
    boats++;
}
```

Pattern 5: Job Sequencing with Deadlines

You have jobs that give a profit if completed before a deadline. Each job takes 1 unit of time.

- **Why we use it:** To maximize profit when time slots are the constraint.
- **How to identify:**
 - Input: Profit, Deadline.
 - Constraint: Each job takes 1 unit of time.
- **The Strategy:**
 1. Sort jobs by **Profit (Descending)**.
 2. Schedule the job as late as possible (closest to its deadline) to leave early slots open for other jobs that might have earlier deadlines.

Java

```
// 1. Sort by Profit Descending
Arrays.sort(jobs, (a, b) -> b.profit - a.profit);

boolean[] slots = new boolean[maxDeadline + 1];
int maxProfit = 0;

for (Job job : jobs) {
    // 2. Find a free slot from deadline down to 1
    for (int j = job.deadline; j > 0; j--) {
        if (!slots[j]) {
            slots[j] = true; // Book the slot
            maxProfit += job.profit;
            break;
        }
    }
}
```

Pattern 6: Graph Greedy (Minimum Spanning Tree)

Algorithms like Prim's and Kruskal's are inherently greedy.

- **Why we use it:** To connect a graph with the minimum total edge weight.
- **How to identify:**
 - "Connect all cities/computers/nodes with minimum cost."
- **The Strategy (Kruskal's):**
 1. Sort all edges by weight.
 2. Pick the smallest edge.
 3. If adding it creates a cycle, discard it. Otherwise, include it.
- **The Strategy (Prim's):** Start from a node, always pick the smallest edge connected to the currently visited set of nodes.

Java Snippet (Kruskal's Logic):

Java

```
// 1. Sort edges by weight
Collections.sort(edges, (a, b) -> a.weight - b.weight);

UnionFind uf = new UnionFind(n); // Helper class
int mstCost = 0;

for (Edge e : edges) {
    // 2. Pick smallest edge if it doesn't form a cycle
    if (uf.find(e.src) != uf.find(e.dest)) {
        uf.union(e.src, e.dest);
        mstCost += e.weight;
    }
}
```

Summary Table: How to choose?

Pattern	Keyword / Identifiers	Sorting Criteria
Intervals	Start/End times, "Max non-overlapping"	End Time (Ascending)
Merge Pattern	Merge cost, Connect ropes	Min-Heap (Smallest first)
Knapsack	Value, Weight, "Fraction allowed"	Ratio (Value/Weight Desc)
Partitioning	Matching (Boats, Cookies)	Sort both (Ascending)
Deadlines	Profit, Deadline, "1 unit time"	Profit (Descending)